WEEK 2 · DAY 1 · OVERVIEW

AdaBoost



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Week 1 averaged deep trees grown side by side, and the best of those averages passed Course 4's pruned tree.

The desk now tries the opposite bet: trees too small to be much good alone, one question each, fitted one after another, each told which accounts the trees before it keep getting wrong.

Does that rank the validation accounts as well, how does one round work, where does the attention go, and what is the number the model returns?

The accounts are real, from Taiwan in 2005, so they describe consumer credit there and then, not US credit today.

The four person columns are never features, because a model that used them would sort people by them.

OBJECTIVES

Fit AdaBoost and score it on training and validation rows, then run one round by hand and match the library.

Follow where the row weights go, read the rounds, the learning rate and the member's depth from folds inside the training rows, take the model apart one column at a time, and read what its output is.

ABOUT 50 MIN WITH EXERCISES / 12 SECTIONS / 4 EXERCISES

Your Route Through Day 1

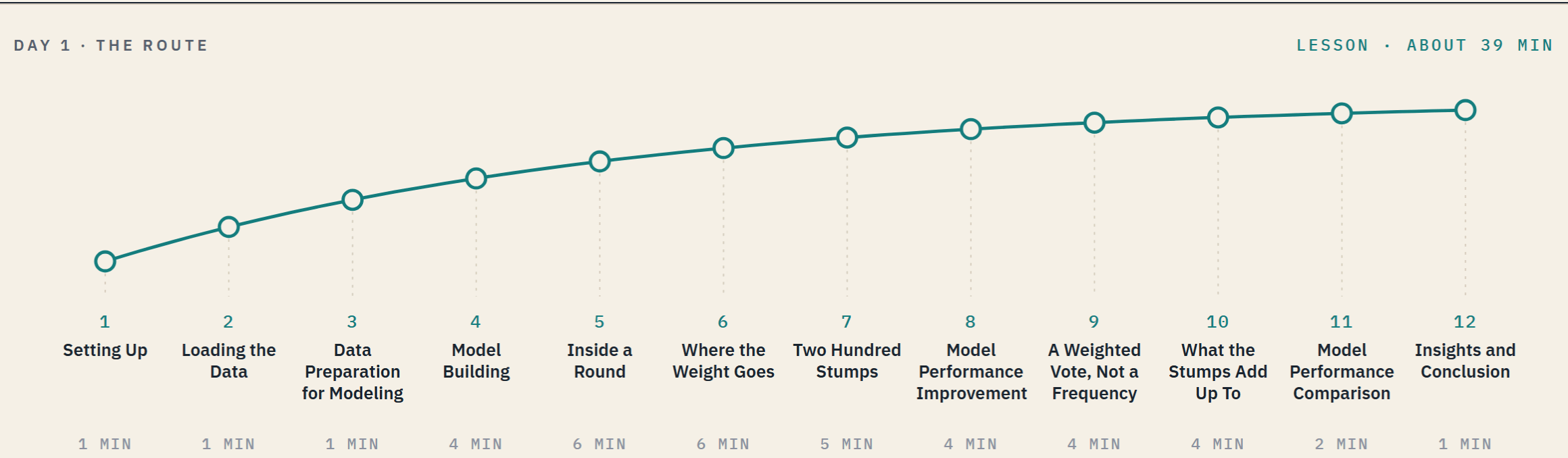


FIGURE 1 Twelve sections, in order, with about how long each takes.

Time. About 39 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
```

Read the card file and pick 19 columns.

Loading the Data

THE FILE'S COLUMNS

- **LIMIT_BAL**: credit limit, NT dollars
- **PAY_0**, **PAY_2** to **PAY_6**: repayment status, September back to April 2005, where -1 is paid on time, 1 to 9 is months late, and -2 and 0 are undocumented (used as numbers, a modelling choice, as in Course 4)
- **BILL_AMT1** to **BILL_AMT6**: the bill, NT dollars, September back to April
- **PAY_AMT1** to **PAY_AMT6**: the amount paid, NT dollars, September back to April
- **default payment next month**: 1 if the account defaulted the following month, **the target**
- **ID**, **SEX**, **EDUCATION**, **MARRIAGE**, **AGE**: never used

READING THE CARD FILE

```
card = pd.read_csv(DATA + "credit_card_default.csv")
TARGET = "default payment next month"
y = card[TARGET]
print(len(FEATURES), "features")
```

```
19 features
```

ROWS AND THE DEFAULT SHARE

```
(30000, 25)
Defaults: 6636  Share: 22.12 percent
```

The file Course 4 used, unchanged, so we rebuild its rows exactly.

Data Preparation for Modeling

COURSE 4'S SPLIT

```
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
X_train, X_valid, y_train, y_valid = train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
print("Training:", len(X_train), " Validation:", len(X_valid), " Test:", len(X_test))
print("Default share:", round(y_train.mean(), 4), round(y_valid.mean(), 4), round(y_test.mean(), 4))
```

```
Training: 18000  Validation: 6000  Test: 6000
Default share: 0.2212 0.2212 0.2212
```

THE THREE KINDS OF ROWS

The same 18,000, 6,000 and 6,000 accounts as Course 4, each with the same default share.

Every model is reported on the validation rows; Course 5 chooses nothing on them, and never scores the test rows.

Training rows fit, validation rows report, test rows stay closed.

Score one stump, then fifty stumps together.

Model Building

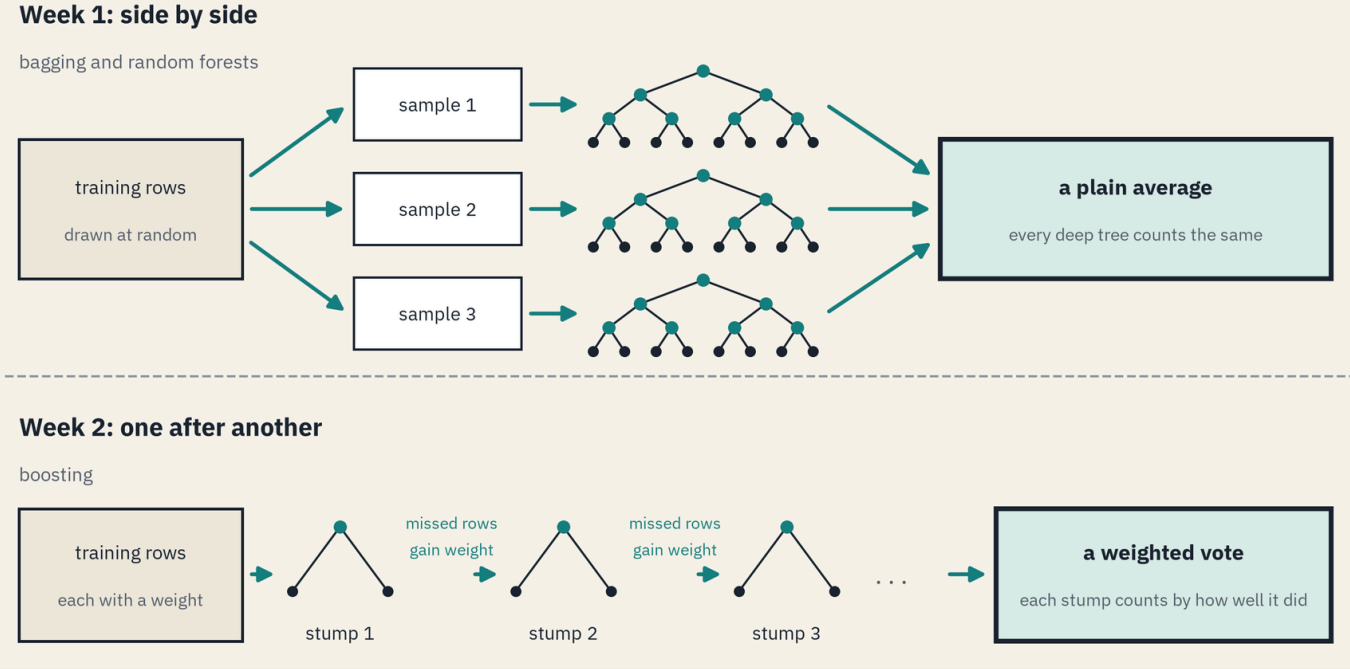


FIGURE 2 Week 1 averages deep trees grown side by side; week 2 fits stumps one after another into a weighted vote.

ONE STUMP

```
Question: PAY_0 <= 1.5
```

	ROC AUC	ACCURACY	RECALL
training	0.6459	0.8209	0.3320
validation	0.6361	0.8160	0.3135

FIFTY STUMPS

```
ada50 = AdaBoostClassifier(random_state=SEED)
```

	ROC AUC	ACCURACY	RECALL
training	0.7760	0.8206	0.3353
validation	0.7672	0.8168	0.3195

Fifty one-question trees rank unseen accounts far better than one. Next: how each round decides what to ask.

Inside a Round

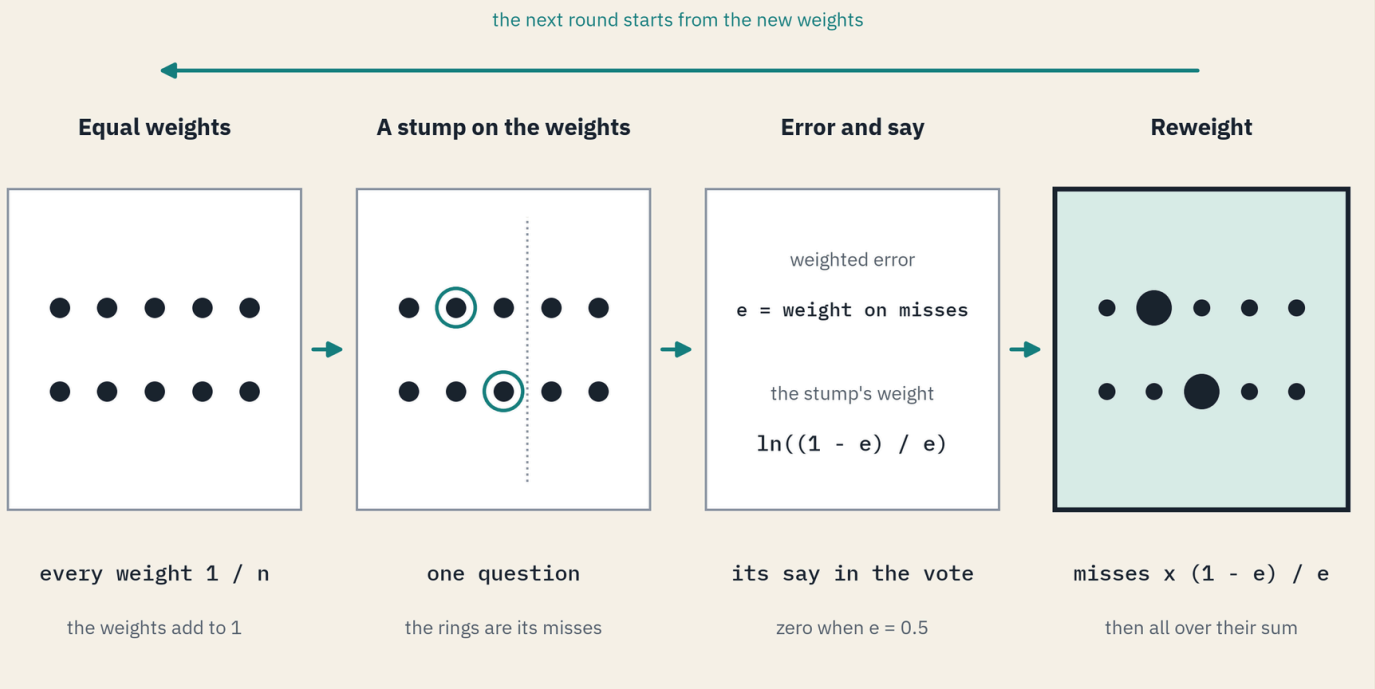


FIGURE 3 One round: equal weights, a stump, its weighted error and weight, then each missed row reweighted.

THREE ROUNDS AGAINST THE LIBRARY

	QUESTION	ERROR	WEIGHT
1	PAY_0 <= 1.5	0.1791	1.5228
2	PAY_AMT1 <= 1223.5	0.4128	0.3526
3	PAY_3 <= 1.5	0.4356	0.2592

THE QUESTION

Q. What share of the weight do they hold after it? Write down a number.

`AdaBoostClassifier` **is these steps, repeated.**

Where the Weight Goes

THE QUESTION

The defaulted accounts are 22.12 percent of the training rows, so they start with 0.2212 of the weight.

Q. After 50 rounds, what share of the weight do they hold? Write down a number.

FIFTY ROUNDS OF WEIGHTS

```
weights = np.full(n, 1 / n)
history = [weights]
for r in range(50):
    member, error, alpha, weights = boost_round(weights)
    history.append(weights)
```

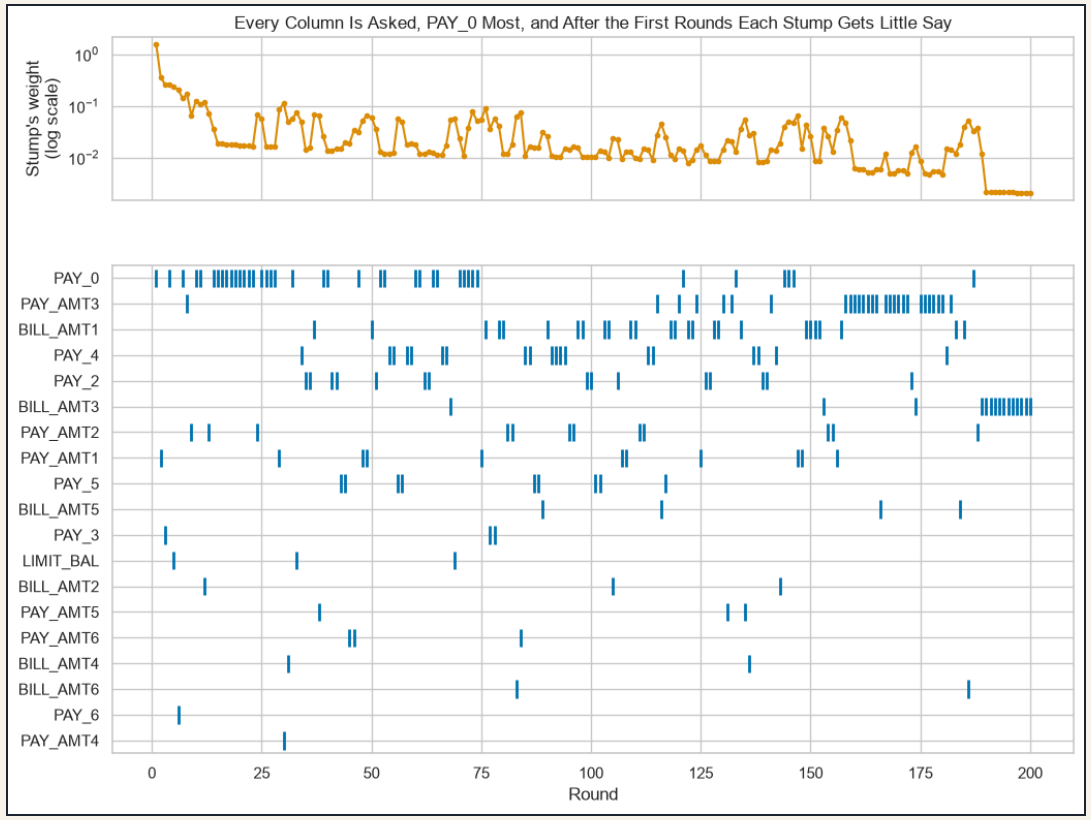
THE WEIGHT MAP'S SLIDER

```
def draw_weights(round_number=0):
    fig, ax = plt.subplots(figsize=(10, 6))
    weight_map(round_number, ax)
    ax.set(xlabel="September status (PAY_0), 3 and above together", ylabel="Credit limit (NT dollars, log scale)")
    ax.legend(loc="lower right")
    plt.show()
interact(draw_weights, round_number=(0, 50, 1));
```

Our 50 rounds match the library's to the last digit a computer keeps.

Fit 200 stumps and score every round.

Two Hundred Stumps



THE COLUMN EACH ROUND ASKS

```
counts = asked.value_counts()
print("Columns asked:", len(counts))
```

```
Columns asked: 19
```

THE LATE ROUNDS

```
Rounds 20 to 200: error 0.4721 to 0.4995  weight 0.0021 to 0.1117
Round one's share of the total weight: 0.1923
```

READING THE PLOT

An **event plot** marks each moment something happens: here, one row per column and a tick at every round that asks it.

All 19 columns are asked: `PAY_0` in 40 rounds, `PAY_AMT3` in 28, `BILL_AMT1` in 26.

Model Performance Improvement

THE QUESTION

Q. After one round, how far apart will the held-out ROC AUC be at learning rates 0.1 and 1.0? Write down a number.

A MEMBER OF ANY DEPTH, AT ANY RATE

```
def boosted(depth=1, rate=1.0, rounds=200):
    return AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=depth, random_state=SEED),
                              n_estimators=rounds, learning_rate=rate, random_state=SEED)
folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
```

DEPTH 1, 2 AND 3 ON THE SAME FOLDS

```
for depth, rounds in [(1, 200), (2, 100), (3, 100)]:
    cv = cross_validate(boosted(depth=depth, rounds=rounds), X_train, y_train, cv=folds, scoring="roc_auc",
                        return_train_score=True, n_jobs=1)
```

Stumps at rate 1.0 for 200 rounds stay the model: the folds chose it, never the validation rows.

A Weighted Vote, Not a Frequency

THE WEIGHTED VOTE

Each stump says yes (+1) or no (-1) about an account.

The **weighted vote** adds every stump's say times its weight and divides by the total weight, so it runs from -1, every stump saying no, to +1.

THE QUESTION

The validation default rate is 0.2212.

Q. What is the average of the model's **`predict_proba`** over the validation accounts? Write down a number.

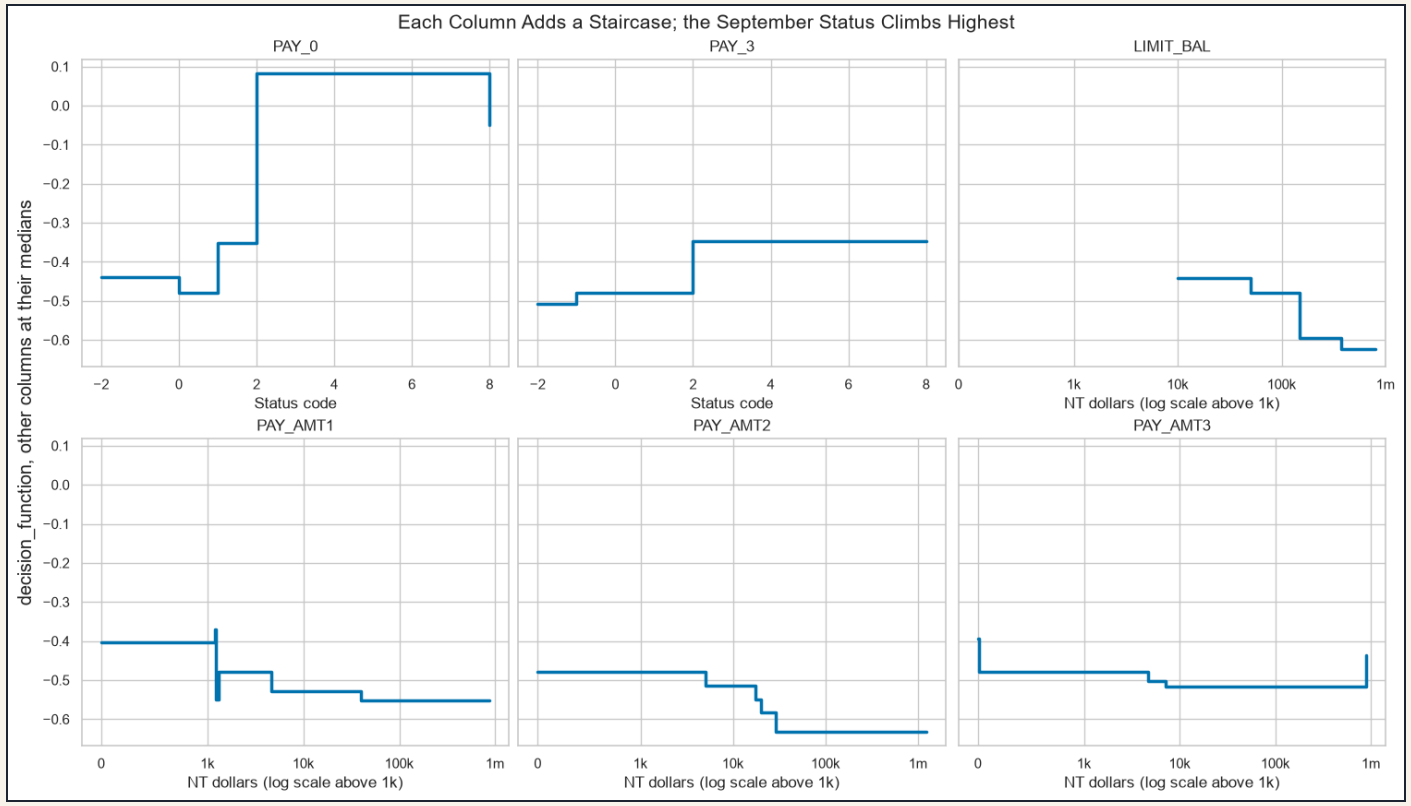
THE VOTE BY HAND, THEN THE LIBRARY'S TWO OUTPUTS

```
says = np.column_stack([2 * m.predict(X_valid.to_numpy()) - 1 for m in ada.estimators_])
vote = says @ ada.estimator_weights_ / ada.estimator_weights_.sum()
```

```
decision = ada.decision_function(X_valid)
score = ada.predict_proba(X_valid)[:, 1]
```

`decision_function` is twice the vote, and `predict_proba` is Course 4's Day 6 logistic curve of it.

What the Stumps Add Up To



PAY_0 SWEPT ACROSS ITS RANGE

```
pd.Series(from_medians, index=values).round(3)
```

```
-2   -0.4380
-1   -0.4380
 0   -0.4790
 1   -0.3530
 2    0.0830
 3    0.0830
 8   -0.0500
```

Two hundred stumps are nineteen staircases added up, and the September status is the tallest.

Score every model on the validation rows.

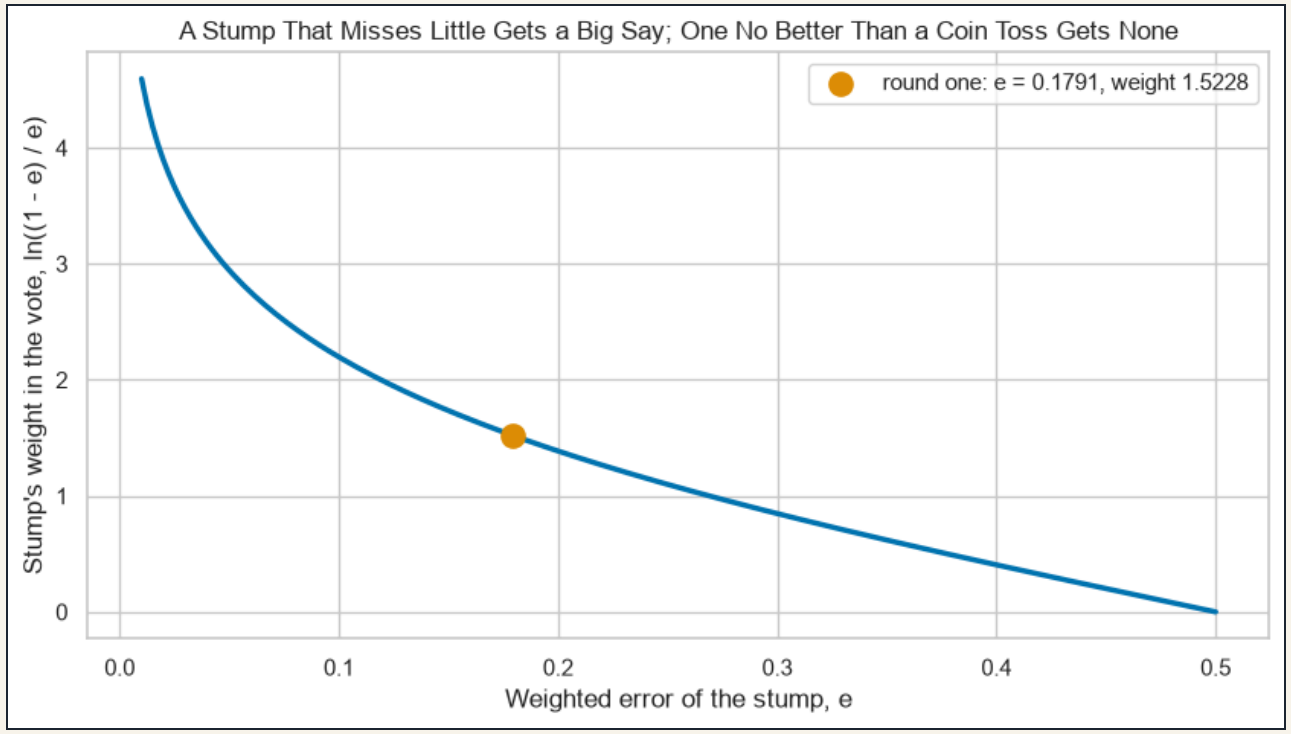
Model Performance Comparison

THE SEVEN MODELS ON THE VALIDATION ROWS

	ROC AUC	ACCURACY	RECALL	PRECISION	F1	AVERAGE PRECISION
One stump	0.6361	0.8160	0.3135	0.6831	0.4298	0.3660
Logistic regression	0.7190	0.8123	0.2442	0.7248	0.3653	0.4946
Default forest	0.7604	0.8150	0.3745	0.6396	0.4724	0.5149
Pruned tree	0.7627	0.7683	0.5840	0.4805	0.5272	0.4954
AdaBoost, 50 stumps	0.7672	0.8168	0.3195	0.6839	0.4355	0.5223
AdaBoost, 200 stumps	0.7708	0.8177	0.3293	0.6817	0.4441	0.5321
Chosen forest	0.7844	0.8192	0.3459	0.6790	0.4583	0.5562

AdaBoost with 200 stumps, 0.7708, passes Course 4's logistic regression and pruned tree and Day 3's default forest; Day 3's chosen forest still leads at 0.7844.

Insights and Conclusion



THE CURVE

```
errors = np.linspace(0.01, 0.5, 200)
plt.plot(errors, np.log((1 - errors) / errors), linewidth=2.5)
```

ROUND ONE'S ERROR AND WEIGHT

```
Weighted error: 0.1791  Stump's weight: 1.5228
```

Day 7 boosts by fitting each tree to what the trees before it still miss.

You Can Now

- Fit **`AdaBoostClassifier`**, read its settings with **`.get_params()`**, and score it on training and validation rows.
- Run one round of AdaBoost by hand: weighted error, the stump's weight, the new row weights.
- Follow where the row weights go and which column each round asks.
- Read the rounds, the learning rate and the member's depth from folds inside the training rows.
- Rebuild the model's output from its weighted vote, and take the model apart one column at a time.

NEXT
WEEK 2 · DAY 2

Gradient Boosting From Residuals

Day 7 boosts by fitting each tree to what the trees before it still miss.